# 프로젝트 1차 리비전 피드백: 물리적 계층 특성 및 기계학습 기반 SIM Box 탐지 시스템 연구 보고서

## 1. 서론 (Introduction)

### 1.1 프로젝트 개요 및 배경

현대 통신 네트워크 환경에서 SIM Box(GSM Gateway)를 이용한 국제전화 우회 사기(Interconnect Bypass Fraud)는 통신 사업자에게 막대한 수익 손실을 야기하고, 망 품질을 저하시키는 주요 위협으로 자리 잡았습니다.[1] 기존의 탐지 체계는 주로 통화 상세 기록(CDR: Call Detail Record) 분석을 통한 '행위 기반(Who)' 또는 '신원 기반' 탐지에 의존해 왔습니다. 그러나 범죄 조직은 수천 개의 SIM 카드를 주기적으로 교체하는 'SIM Rotation' 기술과 인간의 통화 패턴을 모사하는 'HBS(Human Behavior Simulation)' 시스템을 도입하여 이러한 전통적인 탐지망을 손쉽게 우회하고 있습니다.[2]

본 프로젝트는 이러한 한계를 극복하기 위해 관점을 전환하여, 발신자가 '누구인가'가 아닌 패킷이 '어떻게 전송되는가'에 집중하는 물리적 계층(Physical Layer) 기반 탐지 시스템을 제안합니다. SIM Box는 본질적으로 불안정한 사설망과 저성능 하드웨어를 경유하는 게이트웨이 장비이므로, 필연적으로 정상적인 VoLTE 트래픽과는 다른 지연(Latency), 지터(Jitter), 패킷 손실(Packet Loss) 패턴과 같은 '시간적 아티팩트(Timing Artifacts)'를 남길 수밖에 없다는 점에 착안합니다.[1]

### 1.2 1차 리비전 피드백 문서의 목적 및 주요 변경 사항

본 문서는 1차 프로젝트 기획서에 대한 전문가 및 이해관계자의 피드백을 전면 수용하여 작성된 **1차 리비전(v2.0)** 연구 보고서입니다. 1차 리비전 검토 과정에서 제기된 주요 우려 사항은 다음과 같습니다:

1. 탐지 모델의 단순성: 가중치 기반 선형 모델의 높은 오탐(False Positive) 가능성.
2. 성능 병목: Python Scapy 기반 스니핑의 실시간 처리 한계.
3. 데이터 부족: 300세션의 데이터셋이 갖는 통계적 유의성 결여.
4. 개발 일정의 비효율성: 탐지 엔진 개발 착수가 너무 늦어 발생하는 리스크.

이에 따라 본 1차 리비전에서는 고성능 **C++** 패킷 수집 모듈 도입, **ML/DL** 기반의 고도화된 탐지 알고리즘 적용, **SIPp**를 활용한 대규모 트래픽 시뮬레이션, 그리고 **Agile** 방법론에 입각한 병렬 개발 프로세스를 도입하여 시스템의 실효성과 기술적 완성도를 비약적으로 향상시키는 것을 목표로 합니다.

---

# 2. 시스템 아키텍처 고도화: 실시간 처리 성능 최적화

## 2.1 기존 Scapy 기반 설계의 한계점 분석

초기 제안서에서 채택했던 Python 기반의 **Scapy** 라이브러리는 패킷 조작과 프로토콜 분석에는 탁월한 유연성을 제공하지만, 대용량 트래픽을 실시간으로 처리해야 하는 프로덕션 환경의 탐지 시스템으로는 적합하지 않다는 지적이 있었습니다 [User Query].

관련 문헌 및 벤치마크 분석 결과, Scapy는 다음과 같은 치명적인 성능 병목을 가지고 있습니다:

- 객체 생성 오버헤드: Scapy는 수신된 각 패킷을 파이썬 객체로 변환하여 메모리에 적재합니다. VoIP 트래픽과 같이 초당 수천 개(PPS) 이상의 패킷이 유입되는 환경에서는 객체 생성 및 가비지 컬렉션(GC) 부하로 인해 패킷 드랍(Packet Drop)이 발생할 가능성이 매우 높습니다.[3]
- **GIL(Global Interpreter Lock):** 파이썬의 GIL로 인해 멀티스레딩을 활용하더라도 실질적인 병렬 처리가 제한되며, 이는 패킷 캡처와 분석 로직이 CPU 자원을 두고 경쟁하게 만들어 지연 시간(Latency)을 증가시킵니다.[5]

## 2.2 C++ 기반의 하이브리드 아키텍처 제안

이러한 성능 문제를 해결하기 위해, 패킷 수집 및 1차 전처리(Parsing) 단계와 고수준 분석(Analysis) 단계를 분리하는 **C++/Python** 하이브리드 아키텍처를 설계하였습니다.

### 2.2.1 데이터 수집 계층 (Data Ingestion Layer): C++ & PcapPlusPlus

실시간 패킷 캡처의 핵심 성능을 보장하기 위해 C++ 언어를 도입하며, 라이브러리로는 **PcapPlusPlus** 또는 **libtins**를 활용합니다.[7]

- 선정 근거: 벤치마크 결과에 따르면, PcapPlusPlus와 libtins는 libpcap을 래핑하면서도 C++의 효율성을 극대화하여 Scapy 대비 수백 배 이상의 패킷 처리 속도를 보여줍니다.[10] 또한, 타입 안전성(Type Safety)을 제공하여 런타임 오류를 최소화할 수 있습니다.
- **Zero-Copy** 메커니즘: 커널 영역의 소켓 버퍼에서 사용자 영역으로 데이터를 복사하는 오버헤드를 줄이기 위해 PF_RING 또는 최신 리눅스 커널의 AF_XDP 기술과 연동 가능한 C++ 라이브러리를 사용하여 CPU 사용량을 최소화합니다.[11]
- 기능 명세:
  1. 네트워크 인터페이스 카드(NIC)를 프 promiscuous 모드로 설정하여 패킷을 청취합니다.
  2. BPF(Berkeley Packet Filter)를 적용하여 UDP 및 RTP 대역(포트 10000-20000) 트래픽만 선별합니다.
  3. RTP 헤더를 파싱하여 Sequence Number, Timestamp, SSRC, Marker Bit 등의 메타데이터만 추출합니다. (페이로드는 개인정보 보호 및 성능을 위해 제외)

### 2.2.2 데이터 전달 계층 (IPC Layer): Shared Memory

C++에서 추출한 메타데이터를 Python 분석 엔진으로 전달하기 위해, 지연 시간이 거의 없는 공유 메모리(**Shared Memory**) 기법을 사용합니다.

- 구현: POSIX Shared Memory 또는 Redis(인메모리 데이터 스토어)를 큐(Queue)로 활용하여, C++ 프로세스는 데이터를 쓰기(Push)만 하고 Python 프로세스는 읽기(Pop)만 수행하는 비동기 파이프라인을 구축합니다. 이는 소켓 통신이나 파일 입출력 대비 오버헤드를 획기적으로 줄여줍니다.

### 2.2.3 분석 계층 (Analysis Layer): Python & ML Engines

전처리된 수치 데이터가 전달되므로, Python은 무거운 패킷 파싱 작업에서 해방되어 복잡한 통계 연산과 머신러닝 추론(Inference)에 전념할 수 있습니다.

- **Scikit-Learn/XGBoost:** 전달받은 시계열 데이터를 기반으로 특징(Feature)을 추출하고, 사전 학습된 모델을 통해 실시간으로 사기 확률(Risk Score)을 계산합니다.

| 계층 | 기술 스택 **(v1.0)** | 기술 스택 **(v2.0)** | 개선 효과 및 근거 |
|---|---|---|---|
| 패킷 캡처 | Python Scapy | **C++ (PcapPlusPlus)** | 초당 수십만 패킷 처리 가능, 패킷 유실 방지 [10] |
| 데이터 전처리 | Python 객체 변환 | **C++ Struct Parsing** | 메모리 오버헤드 90% 이상 감소 [4] |
| 프로세스 통신 | 단일 프로세스 (함수 호출) | **IPC (Shared Memory)** | 수집과 분석의 병렬 처리, GIL 회피 |
| 분석 엔진 | Python (단순 로직) | **Python (ML/DL Model)** | 복잡한 패턴 인식을 위한 연산 자원 확보 |

---

# 3. 탐지 알고리즘의 고도화: 머신러닝 및 앙상블 기법

## 3.1 가중치 기반 선형 모델의 문제점

1차 리비전에서 제안된 $Score = w_1 \times Jitter + w_2 \times Loss$ 형태의 선형 모델은 구현이 간단하지만, SIM Box가 발생시키는 복잡하고 비선형적인 패턴을 탐지하는 데에는 한계가 명확합니다 [User Query].

- 비선형성 간과: 지터가 낮더라도 패킷 손실이 특정 주기(Burst)로 발생하거나, 시퀀스 번호의 역전 현상이 발생하는 등 다양한 사기 패턴이 존재합니다. 선형 모델은 이러한

복합적인 상관관계를 학습할 수 없습니다.
- 임계값 설정의 어려움**:** 네트워크 환경(LTE, 5G, Wi-Fi)에 따라 정상 트래픽의 기준이 다르므로, 고정된 가중치($w$)는 잦은 오탐(False Positive)이나 미탐(False Negative)을 유발합니다.

## 3.2 다중 머신러닝 모델 도입 전략

본 리비전에서는 오탐 가능성을 획기적으로 줄이고 탐지 정확도를 높이기 위해, 지도 학습(Supervised Learning)과 비지도 학습(Unsupervised Learning)을 결합한 하이브리드 탐지 전략을 수립합니다.[2]

### 3.2.1 지도 학습**: Random Forest & XGBoost (**정확도 중심**)**

SIM Box와 정상 사용자의 라벨링 된 데이터를 이용하여 명확한 사기 패턴을 학습합니다.

- **Random Forest:** 다수의 결정 트리(Decision Tree)를 구성하여 과적합(Overfitting)을 방지하고 노이즈에 강한 모델을 생성합니다. 연구 결과에 따르면 Random Forest는 통신 사기 탐지 분야에서 90% 이상의 높은 정확도를 보였습니다.[12]
  - 주요 특징*:* Feature Importance를 통해 지터, 패킷 간격, 손실률 중 어떤 요소가 사기 판별에 중요한지 해석 가능.
- **XGBoost (Extreme Gradient Boosting):** 이전 트리의 오차를 보정하며 학습하는 부스팅 계열 알고리즘으로, 비정형적인 데이터 분포에서 뛰어난 성능을 발휘합니다. 특히 C++로 작성되어 Python 래퍼를 통해 사용할 때 매우 빠른 추론 속도를 제공하므로 실시간 탐지에 적합합니다.[13]

### 3.2.2 비지도 학습**: Isolation Forest (**제로데이 탐지 중심**)**

기존에 학습되지 않은 새로운 유형의 SIM Box 장비나 네트워크 우회 기법(Zero-day Attack)을 탐지하기 위해 이상치 탐지(Anomaly Detection) 모델을 적용합니다.[14]

- 원리**:** Isolation Forest는 "이상치는 정상 데이터보다 격리(Isolation)되기 쉽다"는 가정을 기반으로 합니다. 데이터를 무작위로 분할했을 때, 정상 데이터는 고립시키기 위해 많은 분할이 필요하지만(트리 깊이가 깊음), 이상치는 적은 분할만으로도 고립됩니다(트리 깊이가 얕음).[16]
- 활용**:** 라벨링 되지 않은 실시간 트래픽을 입력받아 '정상 범주에서 벗어난' 세션을 식별합니다. 이는 지도 학습 모델이 놓칠 수 있는 미지의 위협을 보완하는 역할을 합니다.

### 3.2.3 Rule-based 모델 **(**안전장치**)**

머신러닝 모델의 확률적 오류를 방지하기 위해, 명백한 통신 불량이나 기술적 오류를 걸러내는 규칙 기반 필터를 병행합니다.

- *Rule 1:* RTP Timestamp와 Arrival Time의 편차가 500ms 이상 지속될 경우 (물리적으로 불가능한 지연).
- *Rule 2:* 시퀀스 넘버가 비정상적으로 초기화되거나 건너뛰는 경우.

### 3.3 앙상블 투표 시스템 (Ensemble Voting System)

최종적인 사기 여부 판정은 단일 모델에 의존하지 않고, 위 세 가지 모듈의 결과를 종합하여 결정합니다.

$$FinalDecision = \text{WeightedVote}(\text{RF}, \text{XGBoost}, \text{IsoForest}, \text{Rules})$$

이러한 앙상블 접근법은 개별 모델의 약점을 상호 보완하여 오탐률을 최소화하고 탐지 신뢰도를 극대화합니다.[12]

---

## 4. 데이터셋 구축 전략: SIPp를 활용한 대규모 시뮬레이션

### 4.1 기존 300세션 데이터셋의 한계

머신러닝, 특히 딥러닝이나 앙상블 모델을 학습시키기 위해서는 데이터의 양과 다양성이 필수적입니다. 1차 제안서의 300세션 데이터는 과적합을 유발할 수 있으며, 통계적으로 유의미한 패턴을 도출하기에 턱없이 부족합니다.[2]

### 4.2 SIPp 기반 대규모 트래픽 생성 자동화

VoIP 성능 테스트의 표준 도구인 **SIPp**를 도입하여 데이터셋의 규모를 300세션에서 **10,000**세션 이상으로 확장합니다.[19]

#### 4.2.1 SIPp 시나리오 설계 (XML Scripting)

SIPp의 XML 시나리오 기능을 활용하여 UAC(User Agent Client)와 UAS(User Agent Server) 간의 다양한 통화 흐름을 정의합니다.[20]

- **RTP Playback (pcap play):** 단순한 신호 처리를 넘어, 실제 음성 데이터(RTP)를 포함한 PCAP 파일을 재생(exec play_pcap_audio)하도록 설정합니다.[21] 이는 모든 세션이 동일한 원본 음원을 가지게 하여, 수신 측에서 측정되는 왜곡이 오직 네트워크 조건에 의한 것임을 보장합니다.
- **Call Rate 조절:** 초당 호 생성 수(CPS)를 조절하여, 한적한 시간대부터 폭주하는 시간대까지 다양한 부하 상황을 시뮬레이션합니다.[22]

#### 4.2.2 네트워크 손상 시뮬레이션 (Linux tc netem)

생성된 트래픽에 SIM Box 특유의 물리적 손상을 입히기 위해 Linux 커널의 tc netem (Traffic Control Network Emulator)을 정교하게 설정합니다.[1]

- 시나리오 **A (정상 VoLTE):** 지연 20ms, 지터 2ms (정규분포).
- 시나리오 **B (혼잡한 SIM Box):** 지연 150ms, 지터 50ms, 패킷 손실 1% (Pareto 분포 - 꼬리가 긴 지연 특성 모사).

- 시나리오 **C (**저성능 장비**):** 패킷 재정렬(Reordering) 및 중복(Duplication) 발생.

#### **4.2.3** 데이터셋 구성 목표

| 데이터 유형 | 목표 세션 수 | 생성 도구 | 네트워크 프로파일 |
|---|---|---|---|
| **Normal (Label 0)** | 6,000+ | SIPp + RTP Replay | 고대역폭, 저지연, 낮은 지터 |
| **Fraud (Label 1)** | 4,000+ | SIPp + RTP Replay | 고지연, 버스트 손실, 지터 분산 큼 |
| 총계 | **10,000+** | - | - |

이러한 자동화된 파이프라인을 통해 수집된 대규모 데이터는 Random Forest 및 XGBoost 모델의 학습 안정성을 보장하고, Isolation Forest가 정상 패턴의 범위를 정확히 학습하는 데 기여할 것입니다.

---

# **5.** 프로젝트 일정 및 수행 계획 **(Timeline & R&R)**

## **5.1** 개발 방법론의 전환**: Waterfall**에서 **Agile**로

1차 리비전의 일정은 데이터 수집이 완료된 후 탐지 엔진 개발이 시작되는 순차적(Waterfall) 구조였습니다. 이는 "탐지 엔진 개발 기간이 너무 짧고 늦다"는 지적과 같이, 데이터와 모델 간의 불일치가 발생할 경우 프로젝트 후반부에 치명적인 리스크로 작용할 수 있습니다 [User Query].

따라서, 데이터 수집과 탐지 엔진 개발을 병렬로 진행하며, 초기부터 프로토타입을 지속적으로 검증하는 **Agile/Iterative** 접근 방식으로 전환합니다.

## **5.2** 상세 주차별 계획 **(12**주**)**

### **Phase 1:** 인프라 구축 및 프로토타이핑 **(1~4**주**)**

- **1-2**주차**:** 환경 구축 및 **C++** 스니퍼 개발
  - *Infra:* Asterisk 서버 및 SIPp 트래픽 생성기 설치. 3-Tier 네트워크 토폴로지(Attacker-Gateway-Victim) 구성.
  - *Dev:* PcapPlusPlus 기반의 C++ 패킷 캡처 모듈 프로토타입 개발 (기본적인 RTP 헤더 파싱 및 출력 검증).
  - 핵심 목표*:* Scapy 없이 고속으로 패킷을 유실 없이 캡처할 수 있는지 기술 검증 (PoC).
- **3-4**주차**:** 초기 데이터 생성 및 베이스라인 모델 수립 **(**병렬 진행**)**
  - *Data:* SIPp와 tc netem을 연동하여 '극단적인' 사기/정상 데이터 소량 생성.
  - *Algo:* 생성된 소량 데이터를 이용해 Random Forest 초기 모델 학습. C++ 모듈과

Python 분석기 간의 IPC(공유 메모리) 연동 테스트.
  - 피드백: 초기 모델이 극단적인 케이스를 구분하는지 확인하고, 특징(Feature) 추출 로직 수정.

**Phase 2:** 대규모 데이터 확보 및 모델 고도화 **(5~8**주**)**

- **5-6**주차**:** 대규모 데이터셋 구축 **(SIPp Full-scale)**
  - *Data:* 다양한 시나리오(시간대별, 혼잡도별)를 적용하여 10,000세션 이상의 학습 데이터 생성 및 자동 라벨링.
  - *Algo:* 특징 공학(Feature Engineering) 심화. 단순 지터 외에 '지터의 분산', '패킷 도착 간격의 엔트로피' 등 고차원 특징 개발.
- **7-8**주차**:** 다중 모델 학습 및 비교 평가 **(User Request** 반영**)**
  - *Algo:* Random Forest, XGBoost, Isolation Forest 각각 학습 및 튜닝.
  - *Evaluation:* Rule-based 모델과 ML 모델 간의 성능(Precision, Recall, F1-Score) 비교 분석. 오탐률(False Positive Rate)을 낮추기 위한 앙상블 로직 최적화.

**Phase 3:** 시스템 통합 및 최적화 **(9~12**주**)**

- **9-10**주차**:** 실시간 파이프라인 통합 및 대시보드
  - *Dev:* C++ 스니퍼 -> IPC -> Python ML 엔진 -> 대시보드(Web)로 이어지는 전체 데이터 파이프라인 최적화.
  - *UI:* 실시간 탐지 결과 및 지터 추이 시각화.
- **11-12**주차**:** 종합 테스트 및 최종 보고
  - *Test:* 실제 트래픽 부하 테스트 (Stress Testing) 및 탐지 지연 시간 측정.
  - *Reporting:* 시나리오별 탐지 정확도 및 C++ 최적화 효과에 대한 최종 보고서 작성.

### 5.3 역할 분담 (R&R) 재정의

- 네트워크 시뮬레이션 담당 **(1**명**):** SIPp 스크립트 작성, tc netem 프로파일링, 대규모 데이터셋 생성 및 관리.
- 시스템 개발 담당 **(1**명**):** C++ 패킷 캡처 모듈(PcapPlusPlus) 개발, IPC 및 전체 파이프라인 아키텍처 구현.
- 데이터 분석**/AI** 담당 **(2**명**):** 특징 추출, ML 모델(RF, XGB, IsoForest) 학습 및 튜닝, 앙상블 로직 구현.

---

## 6. 결론 (Conclusion)

본 1차 리비전 연구 보고서는 SIM Box 탐지 시스템의 실효성을 확보하기 위해 필수적인 기술적 도약을 담고 있습니다.

1. **C++** 하이브리드 아키텍처를 통해 Python의 성능 한계를 극복하고 실시간성을 확보했습니다.
2. 머신러닝 앙상블 모델을 도입하여 단순 선형 모델의 오탐 문제를 해결하고 정교한 탐지 로직을 구축했습니다.

3. **SIPp** 기반 시뮬레이션으로 데이터셋의 규모를 30배 이상 확장하여 통계적 신뢰성을 확보했습니다.
4. **Agile** 프로세스를 통해 탐지 엔진 개발을 조기에 착수함으로써 프로젝트 리스크를 최소화했습니다.

이러한 개선 사항들은 본 시스템이 단순히 이론적인 연구에 그치지 않고, 실제 통신망 환경에서 동작 가능한 수준의 성능과 정확도를 갖춘 보안 솔루션으로 발전하는 토대가 될 것입니다.

---

# 7. 의견에 대한 답변 (Responses to Opinions)

**A - "**단순 선형 모델의 오탐 가능성에 대한 우려를 수용하여**, Random Forest, XGBoost, Isolation Forest** 등 다양한 **ML** 모델을 도입하고 **Rule-based**와 성능을 비교하는 실험을 추가함**"**

- 가중치 기반 모델을 폐기하고, 최신 지도/비지도 학습 모델을 적용하여 탐지 정확도를 높였습니다. 또한 7-8주차 일정에 다중 모델 간 성능 비교 평가 단계를 명시적으로 포함시켰습니다.

**A - "Scapy**의 실시간 처리 병목 우려를 수용하여**, C++** 기반 라이브러리**(PcapPlusPlus/libtins)**를 활용한 고성능 패킷 수집 모듈을 추가함**"**

- Python의 성능 한계를 인정하고, 시스템 아키텍처를 C++(수집)와 Python(분석)이 결합된 하이브리드 형태로 재설계하여 실시간 처리 성능을 최적화했습니다.

**A - "300**세션 데이터의 통계적 부족함을 수용하여**, SIPp**를 활용한 대규모 트래픽 생성 및 시뮬레이션을 데이터셋 구축 방안으로 채택함**"**

- 수동 수집 방식 대신 SIPp 툴을 이용한 자동화된 시나리오를 도입하여, 데이터셋 목표를 10,000세션 이상으로 상향 조정했습니다. 이를 통해 ML 모델 학습에 필요한 충분한 데이터를 확보하게 되었습니다.

**A - "**탐지 엔진 개발이 늦다는 지적을 수용하여**,** 데이터 수집과 엔진 개발을 병렬로 진행하는 **Agile** 일정으로 변경하고 초기 프로토타이핑 단계를 앞당김**"**

- 기존의 순차적(Waterfall) 일정을 수정하여, 3주차부터 탐지 엔진 프로토타입 개발을 시작하도록 조정했습니다. 이를 통해 초기 데이터를 이용한 빠른 피드백과 시나리오 수정이 가능하도록 프로세스를 개선했습니다.

참고 자료


1. 프로젝트 1차 리비전_ SIM Box 물리적 특성 기반 탐지 시스템.pdf
2. Simbox Fraud Unmasked – Webinar Recap and Top Questions Answred - Synaptique, 1월 22, 2026에 액세스, https://www.synaptique.com/simbox-fraud-unmasked-webinar-recap-and-top-questions-answred/

3. How to improve slow performance, poor speedup with pypy · Issue #253 · secdev/scapy, 1월 22, 2026에 액세스, https://github.com/secdev/scapy/issues/253
4. Mad-Max Scapy: Improving Scapy's packet sending performance - byt3bl33d3r, 1월 22, 2026에 액세스, https://byt3bl33d3r.github.io/mad-max-scapy-improving-scapys-packet-sending-performance.html
5. Low Performance with Scapy - python - Stack Overflow, 1월 22, 2026에 액세스, https://stackoverflow.com/questions/11348328/low-performance-with-scapy
6. How to improve scapy performance reading large files - Stack Overflow, 1월 22, 2026에 액세스, https://stackoverflow.com/questions/38601091/how-to-improve-scapy-performance-reading-large-files
7. LAN Traffic Capture Applications Using the Libtins Library - MDPI, 1월 22, 2026에 액세스, https://www.mdpi.com/2079-9292/10/24/3084
8. Feature Overview | PcapPlusPlus, 1월 22, 2026에 액세스, https://pcapplusplus.github.io/docs/features
9. Benchmarks - Libtins, 1월 22, 2026에 액세스, https://libtins.github.io/benchmark/
10. Benchmarks | PcapPlusPlus, 1월 22, 2026에 액세스, https://pcapplusplus.github.io/docs/benchmark
11. Is network speed/performance affected by libpcap based application? - Stack Overflow, 1월 22, 2026에 액세스, https://stackoverflow.com/questions/12732819/is-network-speed-performance-affected-by-libpcap-based-application
12. Machine Learning-Based Approach for Identification of SIM Box Bypass Fraud in a Telecom Network Based on CDR Analysis: Case of a Fixed and Mobile Operator in Cameroon - ResearchGate, 1월 22, 2026에 액세스, https://www.researchgate.net/publication/368848374_Machine_Learning-Based_Approach_for_Identification_of_SIM_Box_Bypass_Fraud_in_a_Telecom_Network_Based_on_CDR_Analysis_Case_of_a_Fixed_and_Mobile_Operator_in_Cameroon
13. Machine Learning-Based Approach for Identification of SIM Box Bypass Fraud in a Telecom Network Based on CDR Analysis - SciRP.org, 1월 22, 2026에 액세스, https://www.scirp.org/pdf/jcc_2023022714205981.pdf
14. siForest: Detecting Network Anomalies with Set-Structured Isolation Forest - arXiv, 1월 22, 2026에 액세스, https://arxiv.org/html/2412.06015v1
15. An improved X-means and isolation forest based methodology for network traffic anomaly detection | PLOS One - Research journals, 1월 22, 2026에 액세스, https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0263423
16. Anomaly detection using Isolation Forest - A Complete Guide - Analytics Vidhya, 1월 22, 2026에 액세스, https://www.analyticsvidhya.com/blog/2021/07/anomaly-detection-using-isolation-forest-a-complete-guide/
17. Anomaly Detection with Isolation Forest: A Complete Guide | by Yanivbohbot | Medium, 1월 22, 2026에 액세스, https://medium.com/@yanivbohbot5/anomaly-detection-with-isolation-forest-a-complete-guide-d42da77510a6

18. Comparing Data Mining Classification Algorithms in Detection of Simbox Fraud, 1월 22, 2026에 액세스, https://repository.stcloudstate.edu/msia_etds/17/
19. sipp | Kali Linux Tools, 1월 22, 2026에 액세스, https://www.kali.org/tools/sipp/
20. Welcome to SIPp, 1월 22, 2026에 액세스, https://sipp.sourceforge.net/
21. Handling media with SIPp — SIPp 3.6 documentation, 1월 22, 2026에 액세스, https://sipp.readthedocs.io/en/latest/media.html
22. StarTrinity SIP Tester™ (call generator, simulator) - VoIP monitoring and testing tool, 1월 22, 2026에 액세스, http://startrinity.com/VoIP/SipTester/SipTester.aspx